

# Stochastic optimization

## Stochastic dual dynamic programming

Fabian Bastin

`bastin@iro.umontreal.ca`

Université de Montréal – CIRRELT – IVADO – Fin-ML

## Context

Introduced by Pereira and Pinto (1991), first designed to solve hydro-scheduling problems.

- Stochastic dynamic problem with finite horizon.
- Continuous, finite dimensional state and control.
- Convex cost, additive over stages – we focus here on linear costs.
- Discrete, independent noises.

Julia implementations: the `SDDP.jl` package, and the step-by-step notebook `code/SDDP_from_scratch.ipynb` accompanying these slides.

These slides are based on Anthony Papavasiliou's material,  
<https://ap-rg.eu/wp-content/uploads/2020/05/SDDP.pdf>.

## Problem setting

We consider the multistage stochastic linear program over the horizon  $t = 1, \dots, H$  (we write  $H$ , and not  $T$ , since  $T$  denotes the technology matrix)

$$\min_{x_1 \geq 0} c_1^T x_1 + \mathbb{E} \left[ \min_{x_2 \geq 0} c_2^T x_2 + \mathbb{E} \left[ \dots + \mathbb{E} \left[ \min_{x_H \geq 0} c_H^T x_H \right] \right] \right]$$

subject to  $W_1 x_1 = h_1$  and, for  $t = 2, \dots, H$ ,

$$W_t x_t = h_t(\xi_t) - T_{t-1}(\xi_t) x_{t-1}.$$

Equivalently, in dynamic programming form, with  $Q_{H+1} \equiv 0$ ,

$$V_t(x_{t-1}, \xi_{[t]}) = \min_{x_t \geq 0} \left\{ c_t^T x_t + Q_{t+1}(x_t, \xi_{[t]}) : W_t x_t = h_t(\xi_t) - T_{t-1}(\xi_t) x_{t-1} \right\},$$

$$Q_{t+1}(x_t, \xi_{[t]}) = \mathbb{E} [V_{t+1}(x_t, \xi_{[t+1]}) \mid \xi_{[t]}].$$

The expected cost-to-go is convex and piecewise linear in  $x_t$ : SDDP builds it from below, one supporting hyperplane at a time (background notes). Under stagewise independence, the conditioning disappears and a single  $Q_{t+1}(x_t)$  serves the whole stage.

## The nested L-shaped decomposition subproblem

For each stage  $t = 1, \dots, H$  and node  $k = 1, \dots, N_t$  of the scenario tree, we consider the problem  $NLSD(t, k)$

$$\begin{aligned} \min_{x_{t,k}, \theta_{t,k}} \quad & c_{t,k}^T x_{t,k} + \theta_{t,k} \\ \text{s.t.} \quad & W_t x_{t,k} = h_{t,k} - T_{t-1,k} x_{t-1,a(k)}, \quad (\pi_{t,k}) \\ & D_{t,k,j} x_{t,k} \geq d_{t,k,j}, \quad j = 1, \dots, r_{t,k}, \quad (\rho_{t,k,j}) \\ & E_{t,k,j} x_{t,k} + \theta_{t,k} \geq e_{t,k,j}, \quad j = 1, \dots, s_{t,k}, \quad (\sigma_{t,k,j}) \\ & x_{t,k} \geq 0. \end{aligned} \tag{1}$$

$$E_{t,k,j} x_{t,k} + \theta_{t,k} \geq e_{t,k,j}, \quad j = 1, \dots, s_{t,k}, \quad (\sigma_{t,k,j}) \tag{2}$$

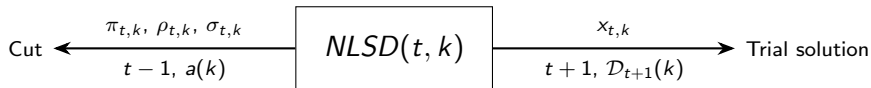
- $N_t$ : number of nodes at stage  $t$ ;  $a(k)$ : ancestor of node  $k$  at stage  $t - 1$ ;  $x_{t-1,a(k)}$ : trial solution received from  $a(k)$ ,
- $\theta_{t,k}$ : epigraph variable approximating the cost-to-go at node  $k$ ;  $\theta_{H,k} \equiv 0$ ,
- Constraints (1): feasibility cuts, constraints (2): optimality cuts.

## Nested L-shaped method

Building block:  $NLSD(t, k)$ , the problem at stage  $t$ , node  $k$ .

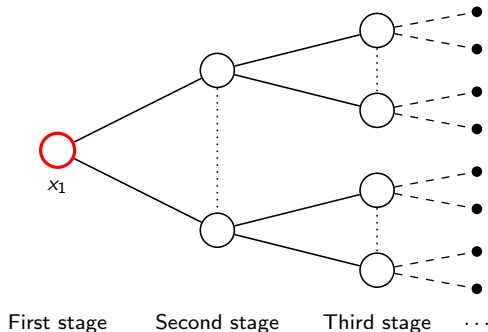
- Repeated application of the L-shaped method.
- Parent nodes send proposals for solutions to their children nodes.
- Child nodes send cuts to their parents.
- There are different sequencing protocols that determine in which order the problems corresponding to different nodes in the scenario tree are solved.

## Nested L-shaped method



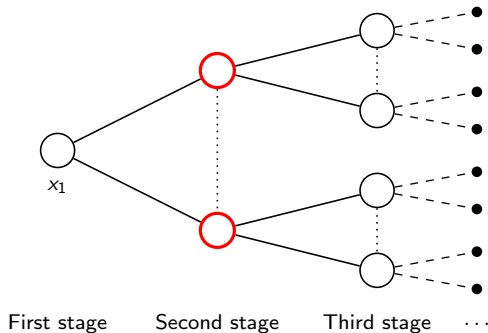
- $a(k)$ : ancestor of node  $k$ .
- $\mathcal{D}_{t+1}(k)$ : descendants of node  $k$  in period  $t + 1$ .

## Illustration: forward pass, stage 1



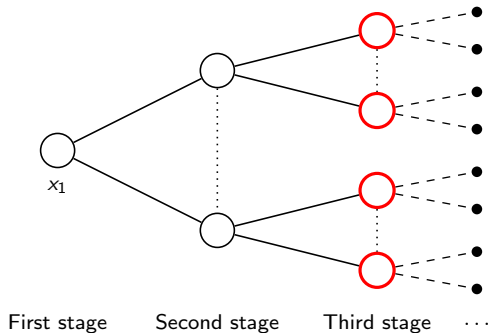
- Node:  $(t = 1, k = 1)$ .
- Direction: forward.
- Output: the trial solution  $x_1$ , sent to every node of stage 2.

## Illustration: forward pass, stage 2



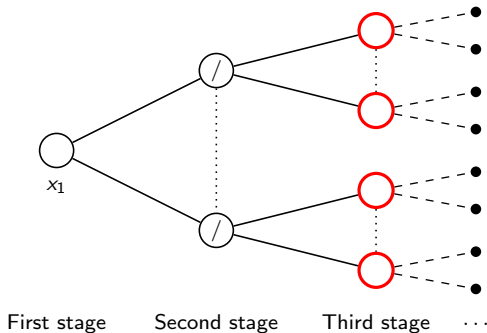
- Node:  $(t = 2, k)$ ,  $k \in \{1, \dots, N_2\}$ .
- Direction: forward.
- Output:  $x_{2,k}$ ,  $k \in \{1, \dots, N_2\}$ .

## Illustration: forward pass, stage 3



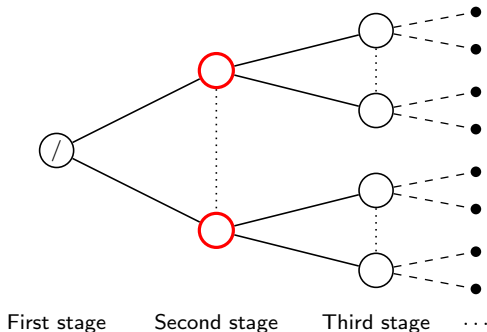
- Node:  $(t = 3, k)$ ,  $k \in \{1, \dots, N_3\}$ .
- Direction: forward.
- Output:  $x_{3,k}$ ,  $k \in \{1, \dots, N_3\}$ .

## Illustration: backward pass, stage 3



- Node:  $(t = 3, k)$ ,  $k \in \{1, \dots, N_3\}$ .
- Direction: backward.
- Output:  $(\pi_{3,k}, \rho_{3,k}, \sigma_{3,k})$ , turned into cuts for the stage-2 nodes (marked /).

## Illustration: backward pass, stage 2



- Node:  $(t = 2, k)$ ,  $k \in \{1, \dots, N_2\}$ .
- Direction: backward.
- Output:  $(\pi_{2,k}, \rho_{2,k}, \sigma_{2,k})$ , turned into a cut for the first-stage problem (marked /).

## Feasibility cuts

If  $NLSD(t, k)$  is infeasible, we compute  $\pi_{t,k}$  and  $\rho_{t,k} \geq 0$  such that

$$\begin{aligned}\pi_{t,k}^T (h_{t,k} - T_{t-1,k} x_{t-1,a(k)}) + \rho_{t,k}^T d_{t,k} &> 0, \\ \pi_{t,k}^T W_t + \rho_{t,k}^T D_{t,k} &\leq 0.\end{aligned}$$

Such a pair is a Farkas certificate of infeasibility, obtained from the dual ray returned by the solver (Farkas' lemma and the extreme rays carrying such certificates are recalled in the linear programming background notes). The following is a valid feasibility cut for  $NLSD(t-1, a(k))$ :

$$D_{t-1,a(k)} x_{t-1,a(k)} \geq d_{t-1,a(k)},$$

where  $D_{t-1,a(k)} = \pi_{t,k}^T T_{t-1,k}$  and  $d_{t-1,a(k)} = \pi_{t,k}^T h_{t,k} + \rho_{t,k}^T d_{t,k}$ .

No feasibility cut is needed when the problem has relatively complete recourse.

## Optimality cuts

For a node  $j$  at stage  $t - 1$ , solve  $NLSD(t, k)$  for every descendant  $k \in \mathcal{D}_t(j)$ , and compute

$$E = \sum_{k \in \mathcal{D}_t(j)} p(k, t | j, t - 1) \pi_{t,k}^T T_{t-1,k},$$
$$e = \sum_{k \in \mathcal{D}_t(j)} p(k, t | j, t - 1) \left( \pi_{t,k}^T h_{t,k} + \sum_{i=1}^{r_{t,k}} \rho_{t,k,i} d_{t,k,i} + \sum_{i=1}^{s_{t,k}} \sigma_{t,k,i} e_{t,k,i} \right),$$

then add the cut  $Ex_{t-1,j} + \theta_{t-1,j} \geq e$  to  $NLSD(t - 1, j)$ .

- $\mathcal{D}_t(j)$ : descendants at stage  $t$  of a node  $j$  at period  $t - 1$ ;
- conditional probabilities:  $p(k, t | j, t - 1) = p_{k,t}/p_{j,t-1}$ .

## Optimality cuts: interpretation

By linear programming duality (background notes), the optimal value of  $NLSD(t, k)$  evaluated at the trial solution  $\hat{x}_{t-1}$  is

$$Q_{t,k}(\hat{x}_{t-1}) = \pi_{t,k}^T (h_{t,k} - T_{t-1,k} \hat{x}_{t-1}) + \rho_{t,k}^T d_{t,k} + \sigma_{t,k}^T e_{t,k},$$

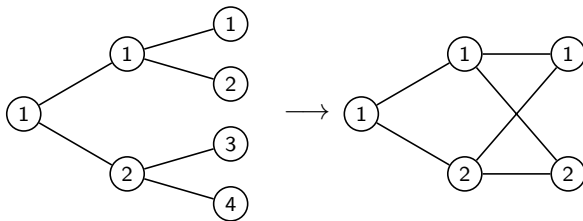
so that, taking the expectation over the descendants,  $e - E\hat{x}_{t-1} = Q_t(\hat{x}_{t-1})$  and  $-E \in \partial Q_t(\hat{x}_{t-1})$ .

The cut  $\theta_{t-1} \geq e - E\hat{x}_{t-1}$  is therefore the supporting hyperplane of the expected cost-to-go at the trial point:

- it is **valid** everywhere, since  $Q_t$  is convex;
- it is **tight** at  $\hat{x}_{t-1}$ .

The approximation is thus refined exactly where the trial solutions are located.

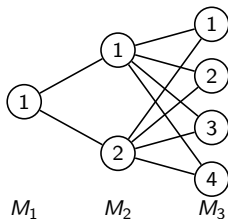
## Recombined scenario tree



- When can we recombine nodes?
- When can we assign the same value function  $Q_{t+1}(x)$  to each node  $k$  of stage  $t$ ?

## Nested decomposition is non-scalable

Assume  $H$  time steps,  $M_t$  discrete outcomes in each stage, and relatively complete recourse (no feasibility cut needed). The tree then has  $N_t = \prod_{j=1}^t M_j$  nodes at stage  $t$ .



- Forward pass:  $M_1 + M_1 \times M_2 + \dots = \sum_{t=1}^H \prod_{j=1}^t M_j$  linear programs;
- Backward pass:  $\sum_{t=2}^H \prod_{j=1}^t M_j$  linear programs.

The effort grows **exponentially** with the horizon.

## Nested decomposition vs extensive form

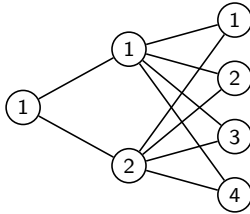
The alternative to nested decomposition is the extensive form.

- The extensive form will not even load in memory.
- Nested decomposition will load in memory, but will not terminate (for large problems).

Nested decomposition lays the foundation for SDDP.

As the method name suggests, we are in the context of dynamic programming.

## Enumerating versus simulating



- Enumeration:  $\{(1, 1), (1, 2), (1, 3), (1, 4), (2, 1), (2, 2), (2, 3), (2, 4)\}$ ;
- Simulation (with 3 samples):  $\{(1, 3), (2, 1), (1, 4)\}$ .

## Making nested decomposition scalable

Solution for the forward pass:

- in the forward pass, we simulate instead of enumerating;
- this results in a probabilistic upper bound and termination criterion.

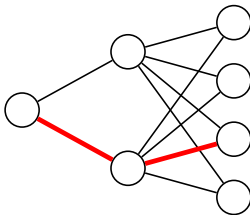
Solution for the backward pass:

- in the backward pass, we share cuts among nodes of the same time period;
- this requires an assumption of **stagewise independence**.

**Stagewise independence:** the random vector  $\xi_{t+1}$  is independent of  $\xi_{[t]} = (\xi_1, \dots, \xi_t)$  for  $t = 1, \dots, H - 1$  (see for instance A. Shapiro, “Analysis of stochastic dual dynamic programming”, European Journal of Operational Research, 209(1), 2011, pp. 63–72). Here,  $\xi_1$  has a unique realization.

## Implications for the forward pass

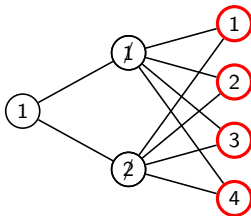
At each forward pass we solve  $H - 1$  subproblems along a single sampled scenario.



For  $K$  Monte Carlo simulations, we solve  $1 + K \times (H - 1)$  linear programs, whatever the size of the scenario tree.

## Implications for the backward pass

Stagewise independence implies the same value function for all nodes of stage  $t \implies$  cut sharing.



For a given trial sequence  $\hat{x}_t$ ,  $t = 1, \dots, H - 1$ , we solve  $\sum_{t=2}^H M_t$  linear programs, and for  $K$  trial sequences,  $K \sum_{t=2}^H M_t$  linear programs: the effort is now **linear** in the horizon.

## Stagewise independence is not necessary

We can use dual multipliers in stage  $t + 1$  for cuts in stage  $t$  even without stagewise independence.

However, each node in stage  $t$  has a different value function, so

- more memory consumption;
- more optimality cuts needed because we are approximating more value functions.

With stagewise independence, we can

- get rid of the scenario tree;
- work with a continuous distribution of  $\xi_t$ .

An intermediate solution consists in adding  $\xi_t$  to the state vector, for instance when the noise follows an autoregressive model.

## SDDP forward pass

- Solve  $NLSD(1, 1)$ . Let  $x_{1,1}$  be the optimal solution. Initialize  $\hat{x}_{1,i} = x_{1,1}$  for  $i = 1, \dots, K$ .
- For  $t = 2, \dots, H$ ,  $i = 1, \dots, K$ ,
  - sample a realization  $\xi_{t,i}$  among  $\xi_{t,k}$ ,  $k = 1, \dots, M_t$ , which determines  $(c_{t,i}, h_{t,i}, T_{t-1,i})$ ;
  - solve  $NLSD(t, i)$  with the trial decision  $\hat{x}_{t-1,i}$ ;
  - store the optimal solution as  $\hat{x}_{t,i}$ , and the stage cost  $c_{t,i}^T \hat{x}_{t,i}$ .

The stage costs exclude  $\theta_{t,i}$ : otherwise the future would be counted twice.

## SDDP backward pass

- For  $t = H, H - 1, \dots, 2$ , for  $i = 1, \dots, K$ ,
  - solve  $NLSD(t, k)$  with the trial decision  $\hat{x}_{t-1,i}$ , for  $k = 1, \dots, M_t$ ;
  - compute

$$E_{t-1} = \sum_{k=1}^{M_t} p_{t,k} \pi_{t,k,i}^T T_{t-1,k},$$
$$e_{t-1} = \sum_{k=1}^{M_t} p_{t,k} \left( \pi_{t,k,i}^T h_{t,k} + \sum_j \rho_{t,k,i,j} d_{t,k,j} + \sum_j \sigma_{t,k,i,j} e_{t,k,j} \right);$$

- add the optimality cut  $E_{t-1}x_{t-1} + \theta_{t-1} \geq e_{t-1}$  to  $NLSD(t-1, k)$ ,  $k = 1, \dots, M_{t-1}$ .

The same cut is added to every node of stage  $t-1$ : this is cut sharing.

## Probabilistic upper bound

Suppose we draw a scenario  $i$ , that is a realization of  $(\xi_t)$ ,  $t = 2, \dots, H$ , and solve  $NLSD(t, i)$  for  $t = 1, \dots, H$  (noting that for  $t = 1$ , the problem is deterministic).

- This gives  $x_{t,i}$ ,  $t = 1, \dots, H$ , with a cost  $z_i = \sum_{t=1}^H c_{t,i}^T x_{t,i}$ .
- Each  $(x_{t,i}, t = 1, \dots, H)$  is feasible but not necessarily optimal, so the expected cost  $\mu = \mathbb{E}[z_i]$  of the current policy is an upper bound on the optimal value.
- Consider  $K$  independent scenarios, and the mean cost  $\bar{z}_K = \frac{1}{K} \sum_{i=1}^K z_i$ . By the Central Limit Theorem,

$$\sqrt{K} (\bar{z}_K - \mu) \xrightarrow{\mathcal{D}} N(0, \sigma^2), \quad \text{i.e., } \bar{z}_K \approx N\left(\mu, \frac{\sigma^2}{K}\right).$$

- $\bar{z}_K$  only estimates that upper bound: a realization of  $\bar{z}_K$  may well fall below the optimal value.

## Bounds and termination criterion

After solving  $NLSD(1, 1)$  in a forward pass we can compute a lower bound  $z_{LB}$  as the objective function value of  $NLSD(1, 1)$ : the cuts are an outer approximation of the expected cost-to-go, hence  $NLSD(1, 1)$  is a relaxation of the original problem.

Terminate if  $z_{LB} \in (\bar{z}_K - \Phi^{-1}(1 - \alpha/2)\hat{s}/\sqrt{K}, \bar{z}_K + \Phi^{-1}(1 - \alpha/2)\hat{s}/\sqrt{K})$ , where  $\Phi(\cdot)$  is the cumulative distribution of a Normal(0,1), and  $\alpha$  is a confidence level set by the user. Typically,  $\alpha = 0.05$ . Since  $z_{LB}$  is a lower bound, only the left inequality really matters.

How to ensure a 1% optimality gap with 95% confidence? Choose  $K$  such that  $\Phi^{-1}(1 - \alpha/2)\hat{s}/\sqrt{K} \approx 0.01\bar{z}_K$ . Using the empirical standard deviation

$\hat{s} = \sqrt{\frac{1}{K-1} \sum_{i=1}^K (z_i - \bar{z}_K)^2}$ , this leads to

$$K = \left( \frac{\Phi^{-1}(1 - \alpha/2)\hat{s}}{0.01\bar{z}_K} \right)^2.$$

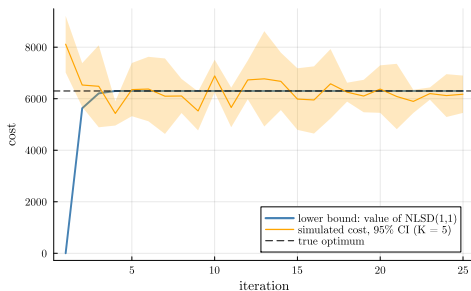
The criterion is optimistic: with a small  $K$  the confidence interval is wide, and the lower bound enters it long before the policy is optimal.

## SDDP algorithm

- 1: Set  $K$ ,  $\alpha$ , the maximum number of iterations, and start with no cut.
  - 2: **repeat**
  - 3:     **Forward pass:** solve  $NLSD(1, 1)$ , set  $z_{LB}$  to its optimal value;
  - 4:     simulate  $K$  scenarios, collect the trial solutions  $\hat{x}_{t,i}$  and the costs  $z_i$ ;
  - 5:     compute  $\bar{z}_K$  and  $\hat{s}$ .
  - 6:     **if**  $z_{LB} \geq \bar{z}_K - \Phi^{-1}(1 - \alpha/2)\hat{s}/\sqrt{K}$  **then**
  - 7:         **stop:** the gap is not statistically significant.
  - 8:     **end if**
  - 9:     **Backward pass:** for  $t = H, \dots, 2$  and  $i = 1, \dots, K$ , solve the  $M_t$
  - 10:     subproblems at  $\hat{x}_{t-1,i}$  and add the resulting cut to stage  $t - 1$ .
  - 11: **until** the iteration limit is reached
- The lower bound is deterministic and non-decreasing; the upper bound is statistical.

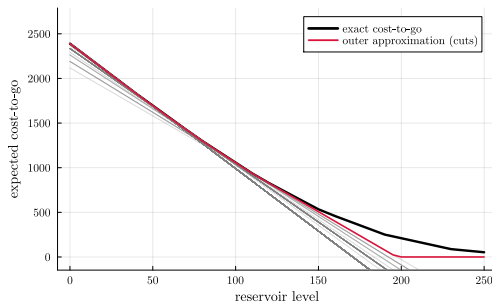
## Numerical illustration

Hydro-thermal scheduling,  $H = 6$  stages,  $M_t = 4$  inflow outcomes (1024 scenarios),  $K = 5$  forward passes per iteration.



The lower bound reaches the optimal value of the extensive form after a few iterations, while the simulated cost keeps oscillating around it.

## Numerical illustration: the cuts



Each cut lies below the exact expected cost-to-go, and the approximation is tight only where the forward passes went: the value function is never learned everywhere. Computed with `code/SDDP_from_scratch.ipynb`.

## References

- M.V.F. Pereira and L.M.V.G. Pinto, “Multi-stage stochastic optimization applied to energy planning”, *Mathematical Programming* 52, 359–375, 1991.
- A. Shapiro, “Analysis of stochastic dual dynamic programming method”, *European Journal of Operational Research* 209(1), 63–72, 2011.
- J.R. Birge and F. Louveaux, *Introduction to Stochastic Programming*, 2nd edition, Springer, 2011 (Chapter 7).
- O. Dowson and L. Kapelevich, “SDDP.jl: a Julia package for stochastic dual dynamic programming”, *INFORMS Journal on Computing* 33(1), 27–33, 2021.
- A. Papavasiliou, lecture notes on SDDP,  
<https://ap-rg.eu/wp-content/uploads/2020/05/SDDP.pdf>.